FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

**Semaine 7**

# Tableau de bord, rapport hebdomadaire et rôles

**Durée : 5 jours** Agrégations SQL · Collections · Dates ISO · Autorisations

## Objectifs de la semaine

- calculer des **indicateurs** (comptages, moyennes) en SQL et avec les collections Laravel ;
- calculer le **MTTR** ;
- manipuler les **semaines ISO** ;
- restreindre des actions selon le **rôle** de l'utilisateur, côté vue **et** côté serveur.

## 1. Agréger : SQL ou collection ?

**En SQL** (MySQL fait le calcul, on ne récupère que le résultat) :

```
Incident::query()
    ->selectRaw('statut, count(*) as total')
    ->groupBy('statut')
    ->pluck('total', 'statut');
// ['ouvert' => 6, 'en_cours' => 2, 'resolu' => 16, 'clos' => 21]
```

**Avec une collection** (les lignes sont chargées en PHP, puis on calcule) :

```
$resolus = Incident::whereNotNull('resolu_at')
    ->where('resolu_at', '>=', now()->subDays(30))
    ->get(['debut_at', 'resolu_at']);

$mttr = $resolus->avg(fn (Incident $i) => $i->dureeMinutes());
```

> **Règle pratique** — Sur des milliers de lignes, faire le calcul en SQL. Sur une centaine de lignes déjà chargées pour l'affichage, une collection est plus lisible.

Compter avec une condition :

```
Site::withCount(['incidents' => fn ($q) => $q->where('debut_at', '>=', now()->subDays(30))])
    ->orderByDesc('incidents_count')
    ->limit(5)
    ->get();
```

## 2. Le MTTR

**MTTR = somme des durées des incidents résolus ÷ nombre d'incidents résolus.**

Exemple : 3 incidents de 30 min, 2 h et 4 h → (30 + 120 + 240) ÷ 3 = **130 min = 2h 10min**.

On n'inclut **que les incidents résolus** : un incident encore ouvert n'a pas de durée finale.

## 3. Les semaines ISO

La norme ISO 8601 numérote les semaines du **lundi au dimanche** ; la semaine 1 est celle qui contient le premier jeudi de l'année. Le champ HTML `<input type="week">` renvoie une valeur du type `2026-W44`.

```
[$annee, $numero] = explode('-W', '2026-W44');
$debut = Carbon::now()->setISODate((int) $annee, (int) $numero)->startOfWeek(); // lundi 00:00
$fin = $debut->copy()->endOfWeek();                                             // dimanche
23:59:59
```

**Piège** — `$debut->endOfWeek()` sans `copy()` **modifie** `$debut` ! Les objets Carbon sont modifiables ; utilise `copy()` ou `CarbonImmutable`.

Un incident « encore ouvert en fin de semaine » est un incident commencé avant la fin de la semaine **et** pas résolu à cette date :

```
->where('debut_at', '<=', $fin)
->where(fn ($q) => $q->whereNull('resolu_at')->orWhere('resolu_at', '>', $fin))
```

## 4. Les autorisations

**Authentification** = qui es-tu ? **Autorisation** = as-tu le droit ?

```
// AppServiceProvider::boot()
Gate::define('admin', fn (User $user) => $user->role === Role::Admin);
```

Trois façons de l'utiliser, à combiner :

```
Route::resource('users', UserController::class)->middleware('can:admin');   // route
Gate::authorize('admin');                                                    // dans une méthode
```

```
@can('admin')
    <a href="{{ route('users.index') }}">Utilisateurs</a>
@endcan
```

**Important** — Cacher un bouton ne protège **rien** : on peut toujours taper l'URL ou envoyer la requête à la main. La vérification côté serveur est obligatoire ; le `@can` dans la vue n'est que du confort.

## Exercices MySQL

1. Nombre d'incidents par statut ( `GROUP BY` ).
2. MTTR global en minutes : `AVG(TIMESTAMPDIFF(MINUTE, debut_at, resolu_at))` sur les incidents résolus.
3. MTTR **par catégorie**, trié du plus long au plus court. Quelle catégorie coûte le plus de temps au NOC ?
4. Nombre d'incidents par semaine ISO : `YEARWEEK(debut_at, 3)` .
5. Top 3 des sites avec le plus d'incidents P1 sur les 30 derniers jours ( `JOIN` , `WHERE` , `GROUP BY` , `ORDER BY` , `LIMIT` ).
6. Comparer ton MTTR SQL avec celui affiché par l'application. Sont-ils identiques ?

## Travail demandé

1. **Tableau de bord** :
   - 4 cartes cliquables avec le nombre d'incidents par statut (lien vers la liste filtrée) ;
   - incidents actifs par sévérité ;
   - MTTR des 30 derniers jours ;
   - top 5 des sites les plus touchés (30 jours) ;
   - P1 en cours avec leur durée, et les 8 derniers incidents.
2. **Rapport hebdomadaire** ( `/rapports/hebdomadaire?semaine=2026-W44` ) :
   - sélecteur de semaine (par défaut la semaine en cours) et période affichée ;
   - indicateurs : incidents de la semaine, résolus, encore ouverts en fin de semaine, MTTR ;
   - répartition par sévérité, par catégorie, par site ;
   - tableau détaillé et liste des incidents encore ouverts ;
   - version imprimable.
3. **Rôles** :
   - seul un admin gère les utilisateurs (mot de passe facultatif en modification) ;
   - seul un admin supprime un incident ;
   - un admin ne peut supprimer ni son propre compte, ni un utilisateur lié à des incidents.

## Livrable

Le tableau de bord est juste, la fiche hebdomadaire s'imprime, les droits sont respectés.

## Auto-évaluation

- [ ] Le MTTR affiché correspond à mon calcul à la main sur 3 incidents
- [ ] Le rapport de la semaine 43 ne contient que les incidents commencés entre le lundi 00:00 et le dimanche 23:59
- [ ] Un incident de la semaine 41 toujours ouvert apparaît dans « encore ouverts » de la semaine 43
- [ ] Un technicien qui tape `/users` reçoit une erreur 403
- [ ] La suppression d'incident est protégée côté serveur, pas seulement cachée
- [ ] Je sais expliquer la différence entre authentification et autorisation

## Ressources

- Laravel — Collections : laravel.com/docs/collections
- Laravel — Autorisation (Gates) : laravel.com/docs/authorization#gates
- MySQL — Fonctions d'agrégation : dev.mysql.com/doc/refman/8.4/en/aggregate-functions.html
- MySQL — YEARWEEK : dev.mysql.com/doc/refman/8.4/en/date-and-time-functions.html

## Pour aller plus loin

Ajouter un graphique (Chart.js via CDN) du nombre d'incidents par jour sur la semaine.